EECE 2323 Digital Logic Design Lab

Name: Arefin Mahmood

Lab Report 4: Adding Data Memory to the Datapath

Date: 06/03/2026

Background & Purpose

In this lab, I added Data Memory to the datapath from Lab 3. In the previous lab, the datapath used the register file and ALU to store values in registers and perform operations. This lab added a larger storage block so values from the register file could be stored in memory and loaded back into the register file later. This is important because processors use memory to store larger amounts of data than registers can hold. The processor datapath in this lab follows a load-store idea, where values are stored from registers into memory and loaded from memory back into registers. In this design, the ALU is used to calculate the memory address, the register file provides the data for store operations, and Data Memory provides the data for load operations.

Design Analysis

The main change from Lab 3 was adding Data Memory and a writeback mux to the datapath. The Data Memory was generated in Vivado using the Distributed Memory Generator IP core. It was configured as a memory with 1024 locations, and each location stores a 16-bit word. Since 1024 locations require 10 address bits, the Data Memory address uses alu_output[9:0]. The memory write data comes from regfile_ReadData2, and the memory output is DataMemOut.

The writeback mux decides what value gets written back into the register file. When MemToReg is 0, the register file writes the ALU output. When MemToReg is 1, the register file writes the Data Memory output. This allows the datapath to support both normal ALU writeback and load operations from memory. I also used VIO to control signals such as RegWrite, MemWrite, MemToReg, ALUSrc1, ALUSrc2, ALUOp, and the register addresses. While testing, I made sure to turn off RegWrite and MemWrite before changing VIO values so I would not accidentally overwrite a register or memory location.

Results and Analysis

I tested the design by writing values into the register file, storing them into Data Memory, loading them back into different registers, and then using the loaded values in an ALU operation. First, I wrote 16'h0004 into register address 2. Then I stored the value from register address 2 into memory address 1. The expected result was DataMemOut = 16'h0004, showing that the store operation worked.

Next, I wrote 16'h0006 into register address 3 and stored the value from register address 3 into memory address 2. The expected result was DataMemOut = 16'h0006. This showed that the datapath could store a second register value into a different memory address. The ALU output acted as the memory address, and regfile_ReadData2 acted as the data being written into memory.

After testing store operations, I tested load operations. I loaded memory address 2 into register address 0, so register 0 received 16'h0006. Then I loaded memory address 1 into register address 1, so register 1 received 16'h0004. For these load operations, MemToReg was set to 1 so the value from DataMemOut was selected as regfile_WriteData.

Finally, I tested whether the loaded values could be used by the ALU. I set ReadAddress1 to 0 and ReadAddress2 to 1. Register 0 contained 16'h0006 and register 1 contained 16'h0004. With

ALUOp set to addition, the ALU output was expected to be 16'h000A. This final result confirmed that the values were stored in memory, loaded back into registers, and used correctly by the ALU.

Conclusion & Recommendations

From this lab, I learned how Data Memory connects to a processor datapath and how load and store operations move data between registers and memory. The important control signals were RegWrite, MemWrite, and MemToReg. RegWrite controls writing into the register file, MemWrite controls writing into Data Memory, and MemToReg controls whether the register file writes the ALU output or DataMemOut. Based on the results, the datapath successfully stored values into memory, loaded them back into registers, and performed an ALU operation using the loaded values. For future testing, I would continue turning RegWrite and MemWrite off before changing VIO values. This avoids accidental writes and makes the hardware test easier to follow.

Appendix A: Prelab and Test Data

Table A1: Lab 4 Testing Sequence

Step	Operation	Important Control Signals	Expected Result
1	Write 16'h0004 into register address 2	RegWrite = 1, MemToReg = 0, ALUSrc1 = 1, ALUSrc2 = 1, ALUOp = 0, WriteAddress = 2, alu_input2_instr_src = 16'h0004	regfile_WriteData = 16'h0004, alu_output = 16'h0004
2	Store register address 2 into memory address 1	MemWrite = 1, ReadAddress2 = 2, ALUSrc1 = 1, ALUSrc2 = 1, ALUOp = 0, alu_input2_instr_src = 16'h0001	regfile_ReadData2 = 16'h0004, alu_output = 16'h0001, DataMemOut = 16'h0004
3	Write 16'h0006 into register address 3	RegWrite = 1, MemToReg = 0, ALUSrc1 = 1, ALUSrc2 = 1, ALUOp = 0, WriteAddress = 3, alu_input2_instr_src = 16'h0006	regfile_WriteData = 16'h0006, alu_output = 16'h0006
4	Store register address 3 into memory address 2	MemWrite = 1, ReadAddress2 = 3, ALUSrc1 = 1, ALUSrc2 = 1, ALUOp = 0, alu_input2_instr_src = 16'h0002	regfile_ReadData2 = 16'h0006, alu_output = 16'h0002, DataMemOut = 16'h0006
5	Load memory address 2 into register address 0	RegWrite = 1, MemToReg = 1, ALUSrc1 = 1, ALUSrc2 = 1, ALUOp = 0, WriteAddress = 0, alu_input2_instr_src = 16'h0002	DataMemOut = 16'h0006, regfile_WriteData = 16'h0006
6	Load memory address 1 into register address 1	RegWrite = 1, MemToReg = 1, ALUSrc1 = 1, ALUSrc2 = 1, ALUOp = 0, WriteAddress = 1, alu_input2_instr_src = 16'h0001	DataMemOut = 16'h0004, regfile_WriteData = 16'h0004
7	Add register address 0 and register address 1	RegWrite = 0, MemWrite = 0, ALUSrc1 = 0, ALUSrc2 = 0, ALUOp = 0, ReadAddress1 = 0, ReadAddress2 = 1	regfile_ReadData1 = 16'h0006, regfile_ReadData2 = 16'h0004, alu_output = 16'h000A

Appendix B: Design Program Files

Appendix B.1: alu_datamem_top.sv

```
`timescale 1ns / 1ps
module alu_datamem_top(
        input clk,
        input logic reset,
        output logic ovf_ctrl,
        output logic take_branch,
        output logic [3:0] disp_en,
        output logic [6:0] seg7_output
    );
    logic [15:0] alu_input1, alu_input2, alu_input2_instr_src;
    logic [15:0] alu_output;
    logic [3:0] ALUOp;
    logic RegWrite;
    logic alu_ovf_flag, alu_take_branch;

    logic [2:0] regfile_ReadAddress1;
```

```
logic [2:0] regfile_ReadAddress2;
logic [2:0] regfile_WriteAddress;
logic [15:0] regfile_WriteData;
logic [15:0] regfile_ReadData1;
logic [15:0] regfile_ReadData2;
logic ALUSrc1, ALUSrc2;
logic [15:0] DataMemOut;
logic [15:0] zero_register = 0;
logic MemWrite;
logic MemToReg;

assign ovf_ctrl = alu_ovf_flag;
assign take_branch = alu_take_branch;

regfile reg1(
    .rst(reset),
    .clk(clk),
    .wr_en(RegWrite),
    .rd0_addr(regfile_ReadAddress1),
    .rd1_addr(regfile_ReadAddress2),
    .wr_addr(regfile_WriteAddress),
    .wr_data(regfile_WriteData),
    .rd0_data(regfile_ReadData1),
    .rd1_data(regfile_ReadData2)
);

TwoToOneMux mux1(
    .a(regfile_ReadData1),
    .b(zero_register),
    .sel_mux(ALUSrc1),
    .out(alu_input1)
);

TwoToOneMux mux2(
    .a(regfile_ReadData2),
    .b(alu_input2_instr_src),
    .sel_mux(ALUSrc2),
    .out(alu_input2)
```

```
);

alu sixteenbit_alu(
    .s(ALUOp),
    .a(alu_input1),
    .b(alu_input2),
    .f(alu_output),
    .ovf(alu_ovf_flag),
    .take_branch(alu_take_branch)
);

data_memory data(
    .clk(clk),
    .we(MemWrite),
    .a(alu_output[9:0]),
    .d(regfile_ReadData2),
    .spo(DataMemOut)
);

TwoToOneMux mux3(
    .a(alu_output),
    .b(DataMemOut),
    .sel_mux(MemToReg),
    .out(regfile_WriteData)
);

Adaptor_display display(
    .clk(clk),
    .input_value(alu_output),
    .disp_en(disp_en),
    .seg7_output(seg7_output)
);

vio_0 vio (
    .clk(clk),
    .probe_in0(regfile_WriteData),
    .probe_in1(regfile_ReadData1),
    .probe_in2(regfile_ReadData2),
```

```
        .probe_in3(alu_input1),
        .probe_in4(alu_input2),
        .probe_in5(alu_take_branch),
        .probe_in6(alu_ovf_flag),
        .probe_in7(alu_output),
        .probe_in8(DataMemOut),
        .probe_out0(RegWrite),
        .probe_out1(alu_input2_instr_src),
        .probe_out2(ALUSrc1),
        .probe_out3(ALUSrc2),
        .probe_out4(ALUOp),
        .probe_out5(MemWrite),
        .probe_out6(MemToReg),
        .probe_out7(regfile_ReadAddress1),
        .probe_out8(regfile_ReadAddress2),
        .probe_out9(regfile_WriteAddress)
    );

endmodule
```

Appendix B.2: regfile.sv

```
`timescale 1ns / 1ps

// it stores 8 values, and each value is 16 bits wide.
module regfile(
    input logic rst,                // reset signal all registers to 0
    input logic clk,                // clock signal
    input logic wr_en,              // rite enable
    input logic [2:0] rd0_addr,     // address for first read port
    input logic [2:0] rd1_addr,     // address for second read port
    input logic [2:0] wr_addr,      // address for write port
    input logic [15:0] wr_data,     // data that will be written into the register file
    output logic [15:0] rd0_data,   // read from rd0_addr
    output logic [15:0] rd1_data    //read from rd1_addr
);

    //creates 8 registers.each register stores 16 bits.
    logic [15:0] registers [0:7];
    integer i;
```

```
    // block handles reset and writing
    // Reset changes right away. Writing only changes on the rising edge of the clock
    always_ff @(posedge clk or posedge rst) begin

        // if reset is on, clear all registers to 0.
        if (rst) begin
            for (i = 0; i < 8; i = i + 1) begin
                registers[i] <= 16'd0;
            end
        end

        // if reset is off and write enable is on, write data.
        else if (wr_en) begin
            registers[wr_addr] <= wr_data;
        end
    end

    // these are the two read outputs. they show the values stored at the selected read addresses.
    assign rd0_data = registers[rd0_addr];
    assign rd1_data = registers[rd1_addr];

endmodule
```

Appendix B.3: TwoToOneMux.sv

```
`timescale 1ns / 1ps

module TwoToOneMux(
    input logic [15:0] a,
    input logic [15:0] b,
    input logic sel_mux,
    output logic [15:0] out
);

    assign out = sel_mux ? b : a;

endmodule
```

Appendix B.4: alu.sv

```
`timescale 1ns / 1ps
```

```
module alu(
    input logic [3:0] s,
    input logic signed [15:0] a,
    input logic signed [15:0] b,
    output logic signed [15:0] f,
    output logic ovf,
    output logic take_branch
);

    always_comb begin
        f = 16'h0000;
        ovf = 1'b0;
        take_branch = 1'b0;

        case (s)
            4'b0000: begin
                f = a + b;
                ovf = (~(a[15] ^ b[15])) & (a[15] ^ f[15]);
            end

            4'b0001: begin
                f = ~b;
            end

            4'b0010: begin
                f = a & b;
            end

            4'b0011: begin
                f = a | b;
            end

            4'b0100: begin
                f = a >>> b;
            end

            4'b0101: begin
```

```
            f = a << b;
        end

        4'b0110: begin
            if (a == 16'h0000) begin
                take_branch = 1'b1;
            end
        end

        4'b0111: begin
            if (a != 16'h0000) begin
                take_branch = 1'b1;
            end
        end

        4'b1000: begin
            f = a ^ b;
        end

        default: begin
            f = 16'h0000;
            ovf = 1'b0;
            take_branch = 1'b0;
        end
      endcase
  end

endmodule
```

Appendix B.5: adaptor_display.sv

```
module Adaptor_display(
  input logic clk,
  input logic [15:0] input_value,
  output logic [3:0] disp_en,
  output logic [6:0] seg7_output
);

  logic refresh_clock;
  logic [1:0] current_display;
```

```
logic [6:0] output_digit;
logic [6:0] digit3, digit2, digit1, digit0;

clockDivider10KHz refreshClock(
    .clk(clk),
    .divided_clk(refresh_clock)
);

displaySelector dSelect(
    .refresh_clk(refresh_clock),
    .dispSelect(current_display)
);

activateDisplay dispEn(
    .dispSelect(current_display),
    .dispID(disp_en)
);

hex_decoder first(
    .clk(clk),
    .hex_digit(input_value[3:0]),
    .segments(digit0)
);

hex_decoder second(
    .clk(clk),
    .hex_digit(input_value[7:4]),
    .segments(digit1)
);

hex_decoder third(
    .clk(clk),
    .hex_digit(input_value[11:8]),
    .segments(digit2)
);

hex_decoder fourth(
    .clk(clk),
```

```
        .hex_digit(input_value[15:12]),
        .segments(digit3)
    );

    bcdSelector bcd_select(
        .x0(digit0),
        .x1(digit1),
        .x2(digit2),
        .x3(digit3),
        .s(current_display),
        .y(output_digit)
    );

    bcdTo7Segment bcd1(
        .x(output_digit),
        .curr_disp(current_display),
        .seg(seg7_output)
    );

endmodule

module clockDivider10KHz(
    input logic clk,
    output logic divided_clk
);

    localparam div_value = 6249;
    integer counter_value = 0;

    always @(posedge clk) begin
        if (counter_value == div_value) begin
            divided_clk <= ~divided_clk;
            counter_value <= 0;
        end
        else begin
            divided_clk <= divided_clk;
            counter_value <= counter_value + 1;
        end
```

```
    end

endmodule

module displaySelector(
    input logic refresh_clk,
    output logic [1:0] dispSelect
);
    always @(posedge refresh_clk)
        dispSelect <= dispSelect + 1;
endmodule

module activateDisplay(
    input logic [1:0] dispSelect,
    output logic [3:0] dispID
);
    always @(dispSelect) begin
        case (dispSelect)
            2'b00: dispID = 4'b1110;
            2'b01: dispID = 4'b1101;
            2'b10: dispID = 4'b1011;
            2'b11: dispID = 4'b0111;
            default: dispID = 4'b1111;
        endcase
    end
endmodule

module hex_decoder(
    input logic clk,
    input logic [3:0] hex_digit,
    output logic [6:0] segments
);
    always @(posedge clk)
        case (hex_digit)
            4'h0: segments = 7'b100_0000;
            4'h1: segments = 7'b1111_001;
            4'h2: segments = 7'b010_0100;
            4'h3: segments = 7'b011_0000;
```

```
            4'h4: segments = 7'b001_1001;
            4'h5: segments = 7'b001_0010;
            4'h6: segments = 7'b000_0010;
            4'h7: segments = 7'b111_1000;
            4'h8: segments = 7'b000_0000;
            4'h9: segments = 7'b001_1000;
            4'hA: segments = 7'b000_1000;
            4'hB: segments = 7'b000_0011;
            4'hC: segments = 7'b100_0110;
            4'hD: segments = 7'b010_0001;
            4'hE: segments = 7'b000_0110;
            4'hF: segments = 7'b000_1110;
            default: segments = 7'b000_000;
        endcase
endmodule

module bcdSelector(
    input logic [6:0] x0,
    input logic [6:0] x1,
    input logic [6:0] x2,
    input logic [6:0] x3,
    input logic [1:0] s,
    output logic [6:0] y
);

    always @(x0, x1, x2, x3, s)
        case (s)
            0: y = x0;
            1: y = x1;
            2: y = x2;
            3: y = x3;
            default: y = 6'bxx;
        endcase

endmodule

module bcdTo7Segment(
    input logic [6:0] x,
```

```
    input logic [1:0] curr_disp,
    output logic [6:0] seg
);
    always @(x, curr_disp) begin
        seg[6] = x[6];
        seg[5] = x[5];
        seg[4] = x[4];
        seg[3] = x[3];
        seg[2] = x[2];
        seg[1] = x[1];
        seg[0] = x[0];
    end
endmodule
```

Appendix B.6: pdatapath.xdc

```
## Clock signal from ENET Controller pin
set_property PACKAGE_PIN H16 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name clk125_pin -period 8.00 -waveform {0 4} [get_ports clk]

## Reset on PYNQ-Z2 board
set_property PACKAGE_PIN D19 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

## LEDs
set_property -dict { PACKAGE_PIN M14 IOSTANDARD LVCMOS33 } [get_ports { ovf_ctrl }]
set_property -dict { PACKAGE_PIN N16 IOSTANDARD LVCMOS33 } [get_ports { take_branch }]

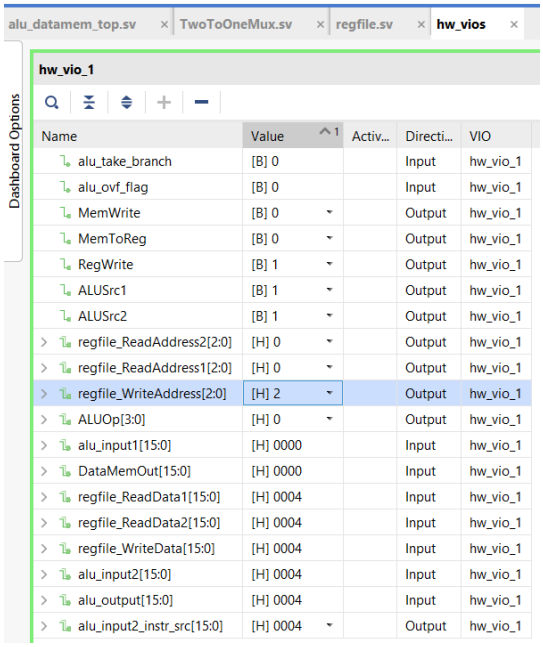
## 7 segment display
set_property -dict { PACKAGE_PIN Y16 IOSTANDARD LVCMOS33 } [get_ports { seg7_output[0] }]
set_property -dict { PACKAGE_PIN Y17 IOSTANDARD LVCMOS33 } [get_ports { seg7_output[1] }]
set_property -dict { PACKAGE_PIN W18 IOSTANDARD LVCMOS33 } [get_ports { seg7_output[2] }]
set_property -dict { PACKAGE_PIN W19 IOSTANDARD LVCMOS33 } [get_ports { seg7_output[3] }]
set_property -dict { PACKAGE_PIN Y18 IOSTANDARD LVCMOS33 } [get_ports { seg7_output[4] }]
set_property -dict { PACKAGE_PIN Y19 IOSTANDARD LVCMOS33 } [get_ports { seg7_output[5] }]
set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports { seg7_output[6] }]

## 7 segment display enable
set_property -dict { PACKAGE_PIN F19 IOSTANDARD LVCMOS33 } [get_ports { disp_en[0] }]
```

```
set_property -dict { PACKAGE_PIN V10 IOSTANDARD LVCMOS33 } [get_ports { disp_en[1] }]
set_property -dict { PACKAGE_PIN V8 IOSTANDARD LVCMOS33 } [get_ports { disp_en[2] }]
set_property -dict { PACKAGE_PIN W10 IOSTANDARD LVCMOS33 } [get_ports { disp_en[3] }]
```

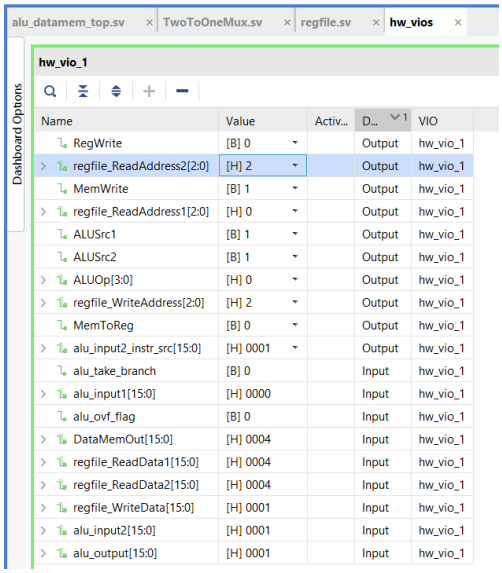
Appendix C: Hardware Evidence

Figure C1: Register write test for register address 2



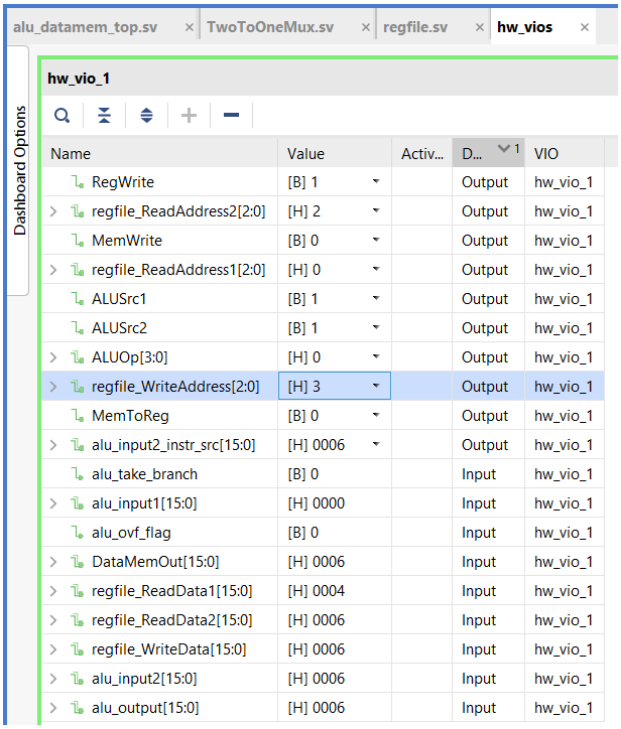
This screenshot shows Step 1. The VIO inputs write 16'h0004 into register address 2. The important outputs are regfile_WriteData = 16'h0004 and alu_output = 16'h0004.

Figure C2: Store register address 2 into Data Memory address 1



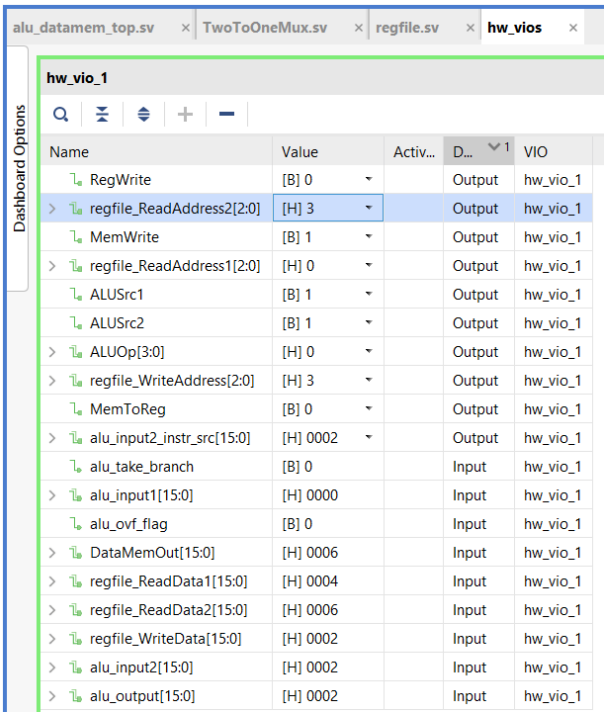
This screenshot shows Step 2. ReadAddress2 is set to register 2, MemWrite is enabled, and the ALU output selects memory address 1. DataMemOut = 16'h0004 confirms the store.

Figure C3: Register write test for register address 3



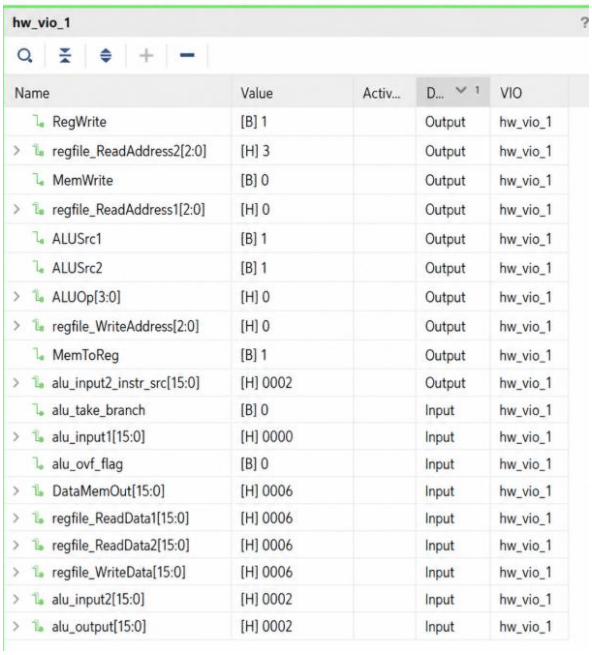
This screenshot shows Step 3. The VIO inputs write 16'h0006 into register address 3. The important outputs are regfile_WriteData = 16'h0006 and alu_output = 16'h0006.

Figure C4: Store register address 3 into Data Memory address 2



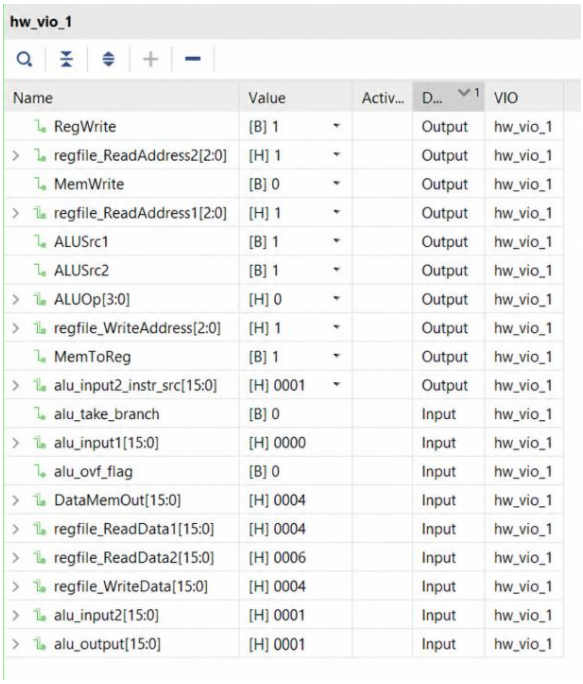
This screenshot shows Step 4. ReadAddress2 is set to register 3, MemWrite is enabled, and the ALU output selects memory address 2. DataMemOut = 16'h0006 confirms the store.

Figure C5: Loading Memory Address 2 into Register Address 0



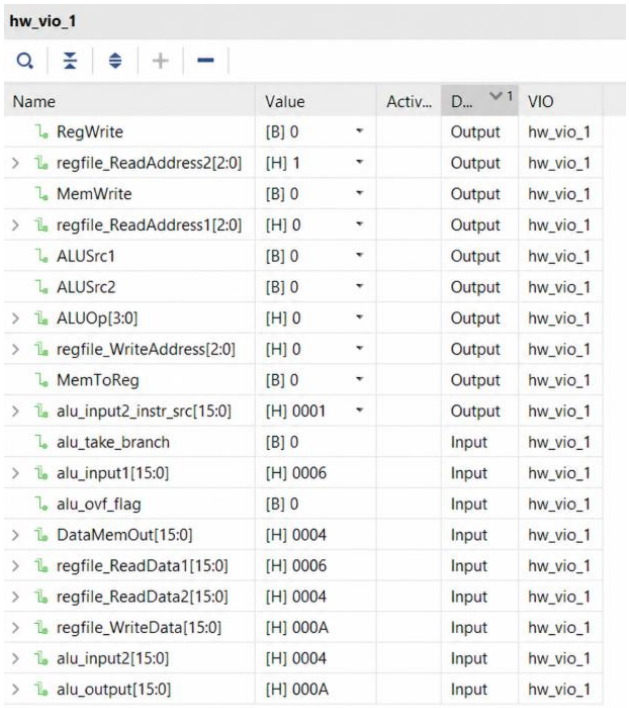
VIO output showing the load operation from Data Memory address 2 into register address 0. The output shows DataMemOut = 16'h0006 and regfile_WriteData = 16'h0006.

Figure C6: Loading Memory Address 1 into Register Address 1



VIO output showing the load operation from Data Memory address 1 into register address 1. The output shows DataMemOut = 16'h0004 and regfile_WriteData = 16'h0004.

Figure C7: Final ALU Add Using Register Addresses 0 and 1



Name	Value	Activ...	D...	VIO
RegWrite	[B] 0		Output	hw_vio_1
regfile_ReadAddress2[2:0]	[H] 1		Output	hw_vio_1
MemWrite	[B] 0		Output	hw_vio_1
regfile_ReadAddress1[2:0]	[H] 0		Output	hw_vio_1
ALUSrc1	[B] 0		Output	hw_vio_1
ALUSrc2	[B] 0		Output	hw_vio_1
ALUOp[3:0]	[H] 0		Output	hw_vio_1
regfile_WriteAddress[2:0]	[H] 0		Output	hw_vio_1
MemToReg	[B] 0		Output	hw_vio_1
alu_input2_instr_src[15:0]	[H] 0001		Output	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1
alu_input1[15:0]	[H] 0006		Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
DataMemOut[15:0]	[H] 0004		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0006		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0004		Input	hw_vio_1
regfile_WriteData[15:0]	[H] 000A		Input	hw_vio_1
alu_input2[15:0]	[H] 0004		Input	hw_vio_1
alu_output[15:0]	[H] 000A		Input	hw_vio_1

VIO output showing the final add operation using register address 0 and register address 1. The output shows regfile_ReadData1 = 16'h0006, regfile_ReadData2 = 16'h0004, and alu_output = 16'h000A.